1. Constitution Validation

The `/speckit.constitution` command was executed once at the beginning of the project in order to define the high-level implementation principles that Codex must follow during the development workflow. This command produced the file `constitution.md`, which establishes the authoritative project sources, architectural constraints, and traceability requirements for the entire Spec-Kit pipeline.

The constitution defines the following constraints:

- the authoritative project sources that must be used to generate the specification
- the architectural style and implementation boundaries
- coding standards and development guidelines
- traceability requirements between specification, planning artifacts, and implementation tasks
- the rule that no requirements may be invented beyond those defined in the authoritative sources

The constitution also incorporates coding style guidance to ensure that generated code remains readable and maintainable. This addresses an issue observed during earlier development iterations where code generated without style constraints became difficult to interpret.

Constitution Validation

The generated `constitution.md` file was manually reviewed to confirm that it correctly reflects the constraints provided to Codex in the `/speckit.constitution` command.

The validation confirmed that:

- all authoritative source documents are correctly referenced
- the architectural constraints match the intended project design
- coding style guidance is included
- the traceability and non-invention rules are clearly enforced

No inconsistencies were identified between the original constitution prompt and the generated constitution file.

Result: PASS — constitution file correctly reflects the implementation principles defined at project initialization.

Branching Strategy Justification

This project was implemented using a single combined planning branch (`main`) rather than the numbered feature branch approach commonly used with Spec-Kit.

This decision was made for two practical reasons.

First, the Blend Colour Game specification contains 61 user stories, which would generate an extremely large set of planning artifacts if expanded independently through multiple feature branches. Running the full Spec-Kit pipeline separately for each feature branch would significantly increase token consumption and regeneration overhead.

Second, many gameplay behaviors in the Blend Colour Game are tightly coupled across player identity, room lifecycle, round progression, scoring, and history tracking. Maintaining a single planning branch ensured that all artifacts — `spec.md`, `plan.md`, `data-model.md`, `contracts/`, and `tasks.md` — remained internally consistent while avoiding unnecessary duplication across branches.

This approach preserves full traceability while keeping the development workflow efficient and manageable.

---

2. Specification Generation and Validation

The `/speckit.specify` command was used to convert the Blend Colour Game project description into a structured specification (`spec.md`) that can be processed by the remainder of the Spec-Kit workflow.

In typical Spec-Kit examples, this command expands short user stories into more detailed specifications and functional requirements. In this project, the input material already contained detailed behavioral descriptions and flows. As a result, the specification generation process primarily involved organizing these flows and extracting formal functional requirements from them.

2(a) Clarification (`/speckit.clarify`)

The optional `/speckit.clarify` stage was executed to identify ambiguities within the generated specification and constitution.

Because the Blend Colour Game specification already contained detailed gameplay flows, relatively few ambiguities were detected. However, several operational rules required clarification in order to ensure deterministic system behavior.

Clarification 1 — Tie-Break Rule

Issue

Multiple players may achieve identical scores or color distances.

Clarification

Ties are resolved using the exact unrounded color distance between the player's submitted color and the target color.

Reasoning

This provides deterministic ranking without introducing additional mechanics such as time-based ordering.

Clarification 2 — Late Joiners During Active Matches

Issue

The specification states that lobbies should be locked once a match begins while also allowing players to join rooms.

Clarification

Players may join a room during an active match but remain in a waiting state until the next match begins.

Reasoning

This preserves stable room membership while preventing disruption to the active match.

Clarification 3 — Single-Player vs Multiplayer Room State

Issue

The specification allows both multiplayer rooms and single-player gameplay.

Clarification

Players must leave a multiplayer room before starting single-player gameplay.

Reasoning

Separating these modes simplifies system state management and prevents conflicts between room state and individual gameplay sessions.

2(b) Specification Validation

After clarifications were applied, the generated `spec.md` file was validated against the authoritative project sources.

Validation confirmed that:

- the behavioral flows described in the source documents were copied into the specification without modification beyond grammar and formatting
- functional requirements were extracted directly from these flows
- no additional gameplay mechanics or system behaviors were introduced
- the clarified rules simply formalize behaviors already implied by the original specification

This validation step ensured that the specification remains fully traceable to the original project materials.

Result: PASS — specification is congruent with the authoritative project description.

---

3. Speckit Plan and Checklists

The `/speckit.plan` command generated the high-level planning artifacts that guide implementation.

The following files were generated:

- `plan.md`
- `research.md`
- `data-model.md`
- `contracts/http-interface.md`
- `contracts/realtime-interface.md`
- `quickstart.md`

These artifacts define the system architecture, data model, and interface contracts required to implement the Blend Colour Game.

3(a) `/speckit.checklist` Reporting

The optional `/speckit.checklist` validation was executed to evaluate the generated planning artifacts against standard software quality checklists.

The checklist evaluated:

- requirement completeness
- requirement clarity
- requirement consistency
- scenario coverage
- edge case coverage
- non-functional requirements

- dependencies and assumptions
- ambiguity and conflict detection

Checklist Results

Most checklist items passed without modification.

Three checklist items were flagged as unresolved:

- measurable performance targets
- observability requirements
- explicit privacy constraints

These items were reviewed against the authoritative project sources. Because the source materials do not define measurable requirements for these areas, they were recorded as justified gaps rather than planning errors.

No changes to the planning artifacts were required.

3(b) Planning Validation

3(b)(i) Architecture and Technology Stack

The architecture defined in `plan.md` was reviewed to ensure that it matches the design constraints defined in the constitution.

The system architecture consists of:

- React + TypeScript frontend
- Django backend
- HTTP APIs for authentication and room operations
- WebSocket connections for realtime gameplay synchronization

This architecture preserves the separation between frontend presentation logic and backend authoritative game state management.

Result: PASS

3(b)(ii) Data Model Validation

The entities defined in `data-model.md` were validated against the functional requirements in `spec.md`.

The data model includes entities supporting:

- player identities
- sessions
- multiplayer rooms
- match lifecycle
- round submissions
- scoring
- score history

The entity relationships support all gameplay flows defined in the specification.

Result: PASS

3(b)(iii) Interface Contract Validation

The HTTP and WebSocket contracts were validated against the functional requirements.

HTTP interfaces support:

- authentication
- session management

- room creation and joining
- profile access
- match history retrieval

WebSocket interfaces support realtime events including:

- player join and leave
- match start
- round progression
- submission handling
- scoring updates
- result publication

All interface behaviors correspond directly to the functional requirements defined in `spec.md`.

Result: PASS

3(b)(iv) ER Diagram Consolidation

The assignment instructions require that all `data-model.md` files across branches be merged into a single entity-relationship diagram.

Because this project used a single planning branch, only one data model exists. Therefore no multi-branch consolidation was required.

Entity-Relationship Diagram:

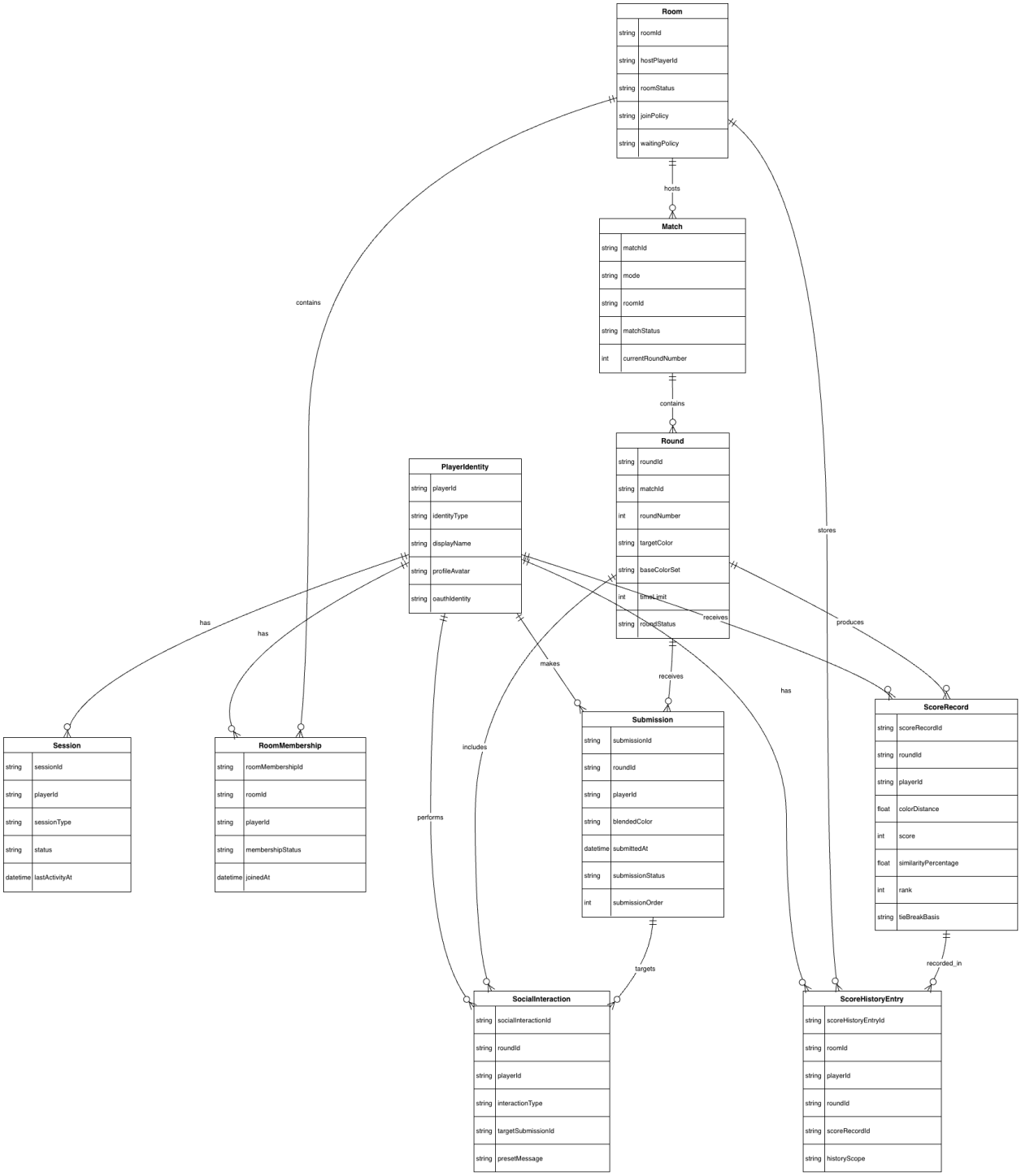

Figure 1 – Entity-Relationship Diagram for the Blend Colour Game Data Model

The diagram directly represents the entities and relationships defined in `data-model.md`.

---

4. Task Generation and Analysis Validation

The `/speckit.tasks` command was used to convert the implementation plan into individual development tasks suitable for execution by Codex.

4(a) Task Generation (`/speckit.tasks`)

The Blend Colour Game specification contains 61 user stories, which would normally produce an extremely large number of tasks if expanded individually.

To keep the task list manageable while maintaining full coverage of the specification, tasks were organized by implementation phases rather than individual user stories.

The resulting task list contained 58 tasks distributed across the following phases.

Phase	Description
Phase 1	Project setup
Phase 2	Backend data models
Phase 3	Gameplay services
Phase 4	HTTP API endpoints
Phase 5	WebSocket realtime communication

Phase 6	Frontend components
Phase 7	System integration
Phase 8	Automated testing

All implementation tasks reference the functional requirements (`FR-XXX`) extracted from the specification, ensuring explicit traceability between specification and implementation.

4(b) Analysis Validation (`/speckit.analyze`)

The optional `/speckit.analyze` stage was executed to validate the consistency between the specification (`spec.md`), planning artifacts (`plan.md`), and the task list (`tasks.md`).

The analysis identifies:

- coverage gaps
- inconsistencies
- duplicated requirements
- remaining ambiguities
- conflicts between artifacts

The detailed `/speckit.analyze` reports generated before and after remediation are provided in:

- **Appendix A — Initial Analyze Report**
- **Appendix B — Final Analyze Report**

Actions Taken

The initial analysis report identified two critical issues:

- Missing requirement traceability within the task list
- Constitution alignment regarding clarified rules

The following corrective actions were applied:

- requirement references were added to tasks to restore traceability
- the constitution was amended to allow clarified requirements produced during **`/speckit.clarify`**

After these corrections were applied, `/speckit.analyze` was executed again.

The final analysis report confirmed:

- all functional requirements are covered by tasks
- no blocking issues remain
- no circular task dependencies exist

A manual inspection of the task ordering in `tasks.md` confirmed that there are no blocking dependencies within the task sequence.

Disposition of Remaining Findings

The final `/speckit.analyze` report still identifies several non-critical planning-quality issues. These findings were reviewed but intentionally not modified for the following reasons.

The performance-goal wording in `plan.md` was flagged because qualitative terms such as “synchronized” and “timely” do not define measurable thresholds. However, the authoritative project sources do not specify measurable performance targets. Introducing numeric thresholds would therefore violate the constitution’s rule that new requirements cannot be invented beyond the original project materials. For this reason the qualitative wording was preserved.

The absence of explicit non-functional requirements was also flagged. Similar to the performance issue, the source materials do not define explicit operational, privacy, or observability requirements. Adding them would introduce requirements that are not traceable to the project description. These areas are therefore considered outside the current requirement baseline.

The frontend pages/ directory inconsistency was identified as a minor structural mismatch between the planning description and the task breakdown. Because this discrepancy does not affect implementation correctness or traceability, it was recorded as a minor planning observation rather than a required correction.

The remaining duplication and task granularity warnings represent documentation and task-organization preferences rather than functional defects. They do not affect requirement coverage or system behavior.

Because all functional requirements are fully covered and no critical issues remain, these remaining findings were recorded but not modified.

Result: PASS — all validation checks satisfied.

Appendix A – Initial `/speckit.analyze` Report

Initial `/speckit.analyze` Table Verbatim (Before Fixes)

Specification Analysis Report

The required prerequisite script .specify/scripts/bash/check-prerequisites.sh --json --require-tasks --include-tasks failed because it enforces

numbered feature branches. I analyzed the active project artifact set directly at specs/project/ in read-only mode.

ID	Category	Severity	Location(s)	Summary	Recommendation
C1	Constitution Alignment	CRITICAL	constitution.md:113, spec.md:357, spec.md:420, plan.md:64	The constitution says tie-break behavior, room-to-single-player transitions, room locking vs waiting-state admission, and guest/authenticated persistence differences MUST remain clarification required unless authoritative docs are updated. The spec and plan resolved three of those by project-local clarification.	Either revert those three decisions back to explicit clarification markers in spec/plan/tasks, or amend the constitution in a separate explicit step before implementation.
C2	Constitution Alignment	CRITICAL	constitution.md:139, tasks.md:10, tasks.md:26	The constitution requires task lists to be grouped by traceable user story or requirement source. tasks.md is grouped by architecture phases only and does not include story labels or requirement references per task.	Add explicit requirement/story traceability to each phase or task, or restructure tasks to satisfy the constitution while keeping the phase-based format.
G1	Coverage Gap	HIGH	plan.md:33, tasks.md:172	The plan introduces qualitative performance goals for synchronized updates and timely countdown/result publication, but no task explicitly defines or validates measurable thresholds.	Either make the performance goals source-backed and measurable, or remove/soften them in plan.md so tasks are not missing implied NFR coverage.
U1	Underspecification	HIGH	spec.md:367, spec.md:528, constitution.md:142	The spec has no explicit non-functional requirements section. Security/privacy, observability, and operational quality are only implied through plan wording and constitution validation expectations.	Add source-

backed NFRs if the authoritative docs support them, or explicitly note that those areas remain outside the current requirements baseline. |

| I1 | Inconsistency | MEDIUM | plan.md:115, tasks.md:143 | The planed frontend structure includes pages/, but the tasks never create or populate that

layer and instead route all UI work into containers/components/hooks. | Align the planned directory structure with the actual intended task breakdown,

or add page-level tasks if pages/ is required. |

| D1 | Duplication | MEDIUM | spec.md:182, spec.md:209, spec.md:450 | Single-player versus multiplayer boundaries are expressed in both User Story 7

and User Story 8, and again in FR-039 through FR-045. This is not contradictory, but it creates repeated responsibility boundaries. | Keep one primary

section for mode-boundary rules and cross-reference it from the others. |

| U2 | Underspecification | MEDIUM | tasks.md:161, tasks.md:162 | T046 and T047 are broad integration tasks spanning entire directories, which weakens

the “immediately executable” goal for task granularity. | Split each into smaller wiring tasks by backend interface family and frontend flow family. |

| U3 | Underspecification | LOW | plan.md:25 | Storage is described generically as persistent backend storage, but does not distinguish what is persi

sted versus authoritative runtime state with enough precision for implementation planning. | Make the storage note explicitly mirror the runtime-vs-p

ersistent split already described in research.md/data-model.md. |

Coverage Summary Table:

| Requirement Key | Has Task? | Task IDs | Notes |

|---|---|---|---|

| player-entry-and-session | Yes | T009, T017, T025-T027, T038-T040, T051-T057 | Covers FR-001 to FR-009 |

| room-membership-and-reconnect | Yes | T010, T014, T018, T028, T033, T037, T041, T049, T052-T055 | Covers FR-010 to FR-011 |

| color-blending-interface | Yes | T011, T015, T021, T038, T042, T044, T056-T057 | Covers FR-012 to FR-018 |

| target-scoring-and-ranking | Yes | T011, T015, T021-T022, T035-T036, T042, T044, T052-T057 | Covers FR-019 to FR-026 |

| timed-round-lifecycle | Yes | T011, T020-T022, T034-T036, T042, T048-T049, T052-T057 | Covers FR-027 to FR-033 |

| results-and-feedback | Yes | T011, T022, T036, T042, T044, T055-T057 | Covers FR-034 to FR-037 |

| multiplayer-sync-and-authority | Yes | T010-T011, T018-T022, T032-T037, T041-T042, T049, T052-T057 | Covers FR-038 to FR-042 |

| mode-selection-and-room-entry | Yes | T010, T018-T020, T028, T040, T043, T048-T049, T052-T057 | Covers FR-043 to FR-045 |

| room-governance | Yes | T010, T014, T018, T028, T033, T049, T052-T055 | Covers FR-046 to FR-049 |

| score-history | Yes | T012, T023, T029, T038, T043, T052-T057 | Covers FR-050 to FR-054 |

| submission-awareness | Yes | T011, T021, T035, T042, T045, T054-T057 | Covers FR-055 to FR-058 |

| social-interaction | Yes | T013, T024, T037, T044-T045, T056-T057 | Covers FR-059 to FR-061 |

| performance-and-operational-quality | Partial | T054-T058 | Inferred only from plan wording; not explicit in spec requirements |

Constitution Alignment Issues:

- Resolved ambiguity decisions in spec/plan conflict with the constitution's guardrail that those topics must remain clarification required until

  authoritative sources change.

- tasks.md does not satisfy the constitution's requirement for grouping by traceable user story or requirement source.

Unmapped Tasks:

- T008 documentation/setup support task
- T050 quickstart walkthrough/update task
- T058 validation/checklist rerun task

These are reasonable support tasks, but they do not map to a discrete functional requirement.

Metrics:

- Total Requirements: 61 functional requirements
- Total Tasks: 58
- Coverage %: 100% inferred functional coverage, partial NFR coverage
- Ambiguity Count: 3
- Duplication Count: 1
- Critical Issues Count: 2

Next Actions

- Resolve the two CRITICAL issues before /speckit.implement.
- If you want to keep the three clarification decisions, update the constitution first in a separate explicit constitution amendment.

- Then adjust tasks.md to restore constitution-compliant traceability without losing the phase-oriented format.

- After that, tighten or remove the plan's qualitative performance goals unless you want explicit measurable validation tasks.

Suggested commands/actions:

- Manually refine spec.md / plan.md to restore or justify ambiguity handling

- Manually refine tasks.md to add requirement or story traceability

- Re-run /speckit.analyze after those edits

Would you like me to suggest concrete remediation edits for the top 3 issues?

Appendix B – Final `/speckit.analyze` Report

Final `/speckit.analyze` Table Verbatim (After Fixes)

Specification Analysis Report

The required prerequisite script .specify/scripts/bash/check-prerequisites.sh --json --require-tasks --include-tasks still fails because it enforces

numbered feature branches. I analyzed the active project artifact set directly at specs/project/ in read-only mode.

| ID | Category | Severity | Location(s) | Summary | Recommendation |

|----|----------|----------|----------|---------|----------|

| G1 | Coverage Gap | HIGH | plan.md:33-35, tasks.md:172-187 | The plan still introduces qualitative performance goals such as “synchronized” and

“timely,” but no measurable thresholds or explicit validation tasks exist for them. | Either quantify these goals and add validation tasks, or remove

them from plan.md so they do not imply uncovered NFRs. |

| U1 | Underspecification | MEDIUM | spec.md:539-562, plan.md:27-35 | The spec has measurable success criteria, but no explicit non-functional

requirements section. Operational quality is mostly expressed in plan wording rather than source-backed requirements. | Either leave NFRs explicitly

out of scope, or add a short note clarifying that operational/performance specifics are not yet part of the approved requirement baseline. |

| I1 | Inconsistency | MEDIUM | plan.md:115-121, tasks.md:139-146 | The planned frontend structure includes frontend/src/pages/, but no task creates

or uses a page layer. | Align the structure description with the actual task list, or add a page-layer task if pages/ is intentional. |

| D1 | Duplication | LOW | spec.md:182-231, spec.md:450-465 | Single-player vs multiplayer boundaries are described in User Stories 7 and 8 and again

in FR-039 to FR-045. The phrasing is consistent but repeated. | Keep one primary section for mode-boundary rules and cross-reference it from the

others. |

| U2 | Underspecification | LOW | tasks.md:161-165 | T046 and T047 remain broad directory-level integration tasks compared with the finer-grained

surrounding tasks. | Split them later if you want maximum execution precision, without changing phase order. |

Coverage Summary Table:

| Requirement Key | Has Task? | Task IDs | Notes |

|---|---|---|---|

| player-entry-and-session | Yes | T001-T009, T017, T025-T031, T038-T040, T051-T057 | Covers FR-001 to FR-009 |

| room-membership-and-governance | Yes | T002, T010, T014, T018, T028, T033, T041, T049, T051-T055 | Covers FR-010, FR-011, FR-045 to FR-049 |

| color-blending-and-round-input | Yes | T003, T011, T015, T021, T038, T042, T044-T045, T055-T057 | Covers FR-012 to FR-018 |

| target-scoring-and-results | Yes | T011, T015, T022, T034-T036, T042, T044, T052, T054-T057 | Covers FR-019 to FR-026, FR-034 to FR-037 |

| timed-round-lifecycle | Yes | T011, T020-T021, T034-T035, T042, T049, T052, T054-T057 | Covers FR-027 to FR-033 |

| multiplayer-sync-and-waiting-state | Yes | T001-T002, T010, T020, T032-T037, T041, T049, T052, T054, T057 | Covers FR-038 to FR-042 |

| mode-selection | Yes | T001, T003, T019, T040, T043, T048-T049, T055, T057 | Covers FR-043 to FR-045 |

| score-history | Yes | T006, T012, T023, T029, T038-T039, T043, T051-T057 | Covers FR-050 to FR-054 |

| submission-awareness | Yes | T011, T021, T032, T035, T042, T045, T054-T058 | Covers FR-055 to FR-058 |

| social-interaction | Yes | T013, T024, T032, T037, T044-T045, T051, T056-T058 | Covers FR-059 to FR-061 |

Constitution Alignment Issues:

- None found after the constitution amendment. The prior ambiguity-guardrail conflict is resolved.

- Requirement traceability in tasks.md is now materially present via (FR-XXX) annotations on every task.

Unmapped Tasks:

- None. Every task now maps to at least one functional requirement.

Metrics:

- Total Requirements: 61
- Total Tasks: 58
- Coverage %: 100% of functional requirements have at least one mapped task
- Ambiguity Count: 1
- Duplication Count: 1
- Critical Issues Count: 0

Next Actions

- No CRITICAL blockers remain.
- You can proceed toward implementation, but it would be cleaner to address the remaining HIGH/MEDIUM planning-quality issues first.
- Highest-value cleanup is the performance-goal wording in plan.md; after that, optionally align the frontend pages/ directory mention with the task

  plan.

Suggested actions:

- Manually edit plan.md to quantify or remove qualitative performance goals

- Manually edit plan.md or tasks.md to align the frontend structure description

- Re-run /speckit.analyze after those cleanups if you want a tighter pre-implementation baseline

Would you like me to suggest concrete remediation edits for the top 3 remaining issues?